

Grade 3 Coding: Python While Loops & Break

Let's learn how to repeat actions with while loops!

What Are Loops?

Loops help us **repeat actions** again and again without rewriting the same code over and over.

Imagine pressing a button on a toy over and over—that's a loop! Have you ever done something many times in real life, like jumping or clapping? That's exactly what loops do in coding!

For Loops vs. While Loops

In Python, there are two main types of loops. Let's look at how they differ:

For Loop

Repeats a **set number of times**! You decide exactly *HOW MANY* times it runs before it even starts.

Example: "Do this 5 times."

Like counting your jumps out loud: 1, 2, 3, 4, 5 — and then you're done!

While Loop

Repeats while something is **TRUE**! It keeps going continuously as long as a specific condition is met.

Example: "Keep going while count is less than 5."

Like eating dinner while you are still hungry!

What Is a While Loop?

A `while` loop repeats actions while a specific condition remains true. Think of it like brushing your teeth—you keep brushing while they are still dirty, and you stop the moment they are clean!

The loop checks the condition each time before starting a new round:

- ✓ If the condition is **TRUE** → It repeats the action.
- ✓ If the condition is **FALSE** → It stops the loop entirely.

Conditions in While Loops

A condition is simply something that evaluates to either **TRUE** or **FALSE**.

Real-World Example: Hand Washing

"While your hands are dirty, keep washing them!"

- If hands are dirty → **TRUE** → Keep washing!
- If hands are clean → **FALSE** → Stop washing!

Simple While Loop Code Example

Let's look at how we write a simple counter loop in Python code:

```
count = 1
while count <= 3:
    print("Jump!")
    count = count + 1
```

How Python Executes This Step-by-Step:

1. **Step 1 (Start):** Set the variable `count` to **1**.
2. **Step 2 (Check):** Repeat while `count` is **3** or less (**`count ≤ 3`**).
3. **Step 3 (Action):** Print "Jump!" to the screen.
4. **Step 4 (Change):** Add **1** to `count` so it tracks the loops.

Real-World Analogy: Playing a Game

You keep playing a board game or video game while you haven't won yet! Just like a while loop, the game keeps going while the winning condition is not true. What games have you played where you try again until you win?

Introducing the Break Statement!

The `break` statement is a special superpower that lets us **stop a loop early**, even if the main condition is still true!

Example: "Keep brushing your teeth... but STOP immediately if your timer rings!"

- The loop is running... running... running...

-  **BREAK!** The loop stops right away.
- The `break` keyword tells Python: *"We are completely done here, move to the next part of the program!"*

While Loop with Break Example

Here is how you use a `break` statement inside an interactive Python loop:

```
while True:
    answer = input("Type 'yes' to stop: ")
    if answer == "yes":
        break
    print("Keep going!")
```

- ✓ **Step 1:** `while True:` creates a loop that wants to run forever.
- ✓ **Step 2:** It asks the user for input and saves it in `answer`.
- ✓ **Step 3:** If your answer is exactly "yes", the `break` stops the loop immediately!
- ✓ **Step 4:** Otherwise, it prints "Keep going!" and loops back to ask again.

Common Mistakes: Infinite Loops

What is an **infinite loop**? It's a loop that **never stops running!** This usually happens when:

- You forget to change the variable or condition inside the loop (e.g., forgetting `count = count + 1`).
- You use `while True:` but miss a `break` statement to exit.

Example of a broken loop: `while True: print("Hi!")` with no `break` will make your computer repeat "Hi!" forever until it runs out of memory! *Always make sure your loop has a way to stop!*

Practice Challenges!

Practice Challenge 1

Challenge: Write a while loop that counts from **1** to **5** and prints each number to the screen.

 **Hint:** Use a variable starting at 1, and add 1 to it each time inside the loop!

Practice Challenge 2

Challenge: Use a while loop and a `break` statement to stop running when the user guesses the secret number `7`.

 **Hint:** Use `input()` to ask for a guess, then use an `if` statement paired with `break` to stop the loop when the answer matches `7`.

 **Discussion:** How does `break` help here? What would happen if we didn't include it?

Summary: What We Learned Today

- **Loops** repeat actions over and over so we don't have to rewrite code.
- **While loops** check a condition and repeat as long as that condition stays **true**.
- The **break** statement lets us escape out of a loop instantly when needed.
- Be careful to avoid **infinite loops** by always updating your variables!

Next Steps & Resources

Try making your own while loops at home! You can use kid-friendly platforms to experiment safely:

- **Scratch** (scratch.mit.edu) - Great for block-based loop practice!
- **Trinket** (trinket.io) - Runs Python directly inside your browser with no setup required!
- **Python for Kids** (python.org/for-kids) - Official guides and tutorials.

 **Every coder started exactly where you are today. Keep practicing and exploring!**

Presented by Python Coding Education | www.reallygreatsite.com

Thank you for learning with us!